

UNIDAD 10

ESTRUCTURAS DE DATOS AVANZADAS (COLECCIONES) Y ALGORITMOS

**Programación
CFGS DAW**

Autor: José Miguel Blázquez
2023/2024

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDO

1. Introducción.....	4
2. Estructuras de datos.....	4
2.1 Fundamentos de estructuras de datos.....	6
2.2 Estructuras de datos en Java: Colecciones.....	10
2.3 Listas: la clase LinkedList.....	12
2.4 Colas: la interfaz Queue.....	14
2.5 Pilas: la clase Stack.....	15
2.6 Mapas: la clase HashMap.....	17
2.7 Conjuntos en Java: la interfaz Set.....	19
2.7.1 Conjuntos ordenados.....	19
2.7.2 Aplicaciones a la teoría de conjuntos: unión, intersección y diferencia.....	20
3. Análisis y diseño con estructuras de datos.....	22
4. Complejidad computacional.....	24
5. Algoritmos sobre estructuras de datos.....	27
5.1 Algoritmos de ordenación.....	27
5.1.1 Ordenación por inserción directa.....	27
5.1.2 Ordenación por el método de la burbuja.....	29
5.1.3 Ordenación por QuickSort.....	30
5.1.4 Ordenación por MergeSort.....	31
5.1.5 Ordenación estándar en Java: el método .sort().....	32
5.1.6 Ordenación de objetos con "sort": Comparable y Comparator.....	33
5.2 Algoritmos de búsqueda.....	36
5.2.1 Búsqueda secuencial.....	36
5.2.2 Búsqueda binaria (dicotómica).....	36
5.3 Otros algoritmos.....	37
5.3.1 Técnicas y estrategias avanzadas para diseño de algoritmos.....	37
5.3.2 Algoritmos del camino más corto y recubrimiento mínimo.....	42

UD10. ESTRUCTURAS DE DATOS AVANZADAS, COLECCIONES Y ALGORITMOS

1. INTRODUCCIÓN

Para avanzar en el estudio del campo de la programación, se requiere aprender y dominar los tipos de estructuras de datos más relevantes y los algoritmos que permiten manipularlas.

Las estructuras de datos son el sistema mediante el cual se organizan los datos en la memoria de la aplicación que se está programando, ofreciendo una forma de almacenar y organizar la información para que se pueda usar de manera eficiente. Existen muchas formas de organizar los datos en la memoria; como ya hemos estudiado anteriormente las estructuras estáticas, en esta unidad nos centraremos en las estructuras dinámicas y avanzadas. Abarcan también el conjunto de algoritmos que podemos usar en cualquier lenguaje de programación para estructurar los datos en la memoria.

Estos algoritmos determinan cómo se manipulan y organizan los datos almacenados en las estructuras de datos, definiendo las operaciones que se pueden realizar sobre los datos, como la búsqueda, inserción, eliminación, clasificación y ordenación. Los algoritmos y las estructuras de datos están estrechamente relacionados y se apoyan mutuamente en el procesamiento y la manipulación de datos en las aplicaciones. Una buena comprensión de ambos conceptos es esencial para el desarrollo de soluciones informáticas eficientes y efectivas.

En unidades anteriores hemos trabajado con elementos como arrays o cadenas, junto con tipos básicos de Java (enteros, dobles, etc.). Todos estos se denominan tipos estáticos porque su tamaño se establece cuando declaramos las variables correspondientes y nunca cambia durante la ejecución del programa.

Por otro lado, los tipos dinámicos pueden cambiar su tamaño a lo largo de la ejecución del programa. Por ejemplo, si tenemos una lista de libros, podemos agregar o eliminar libros dinámicamente de la lista, sin tener que aumentar o disminuir manualmente ningún contador, ni reasignar toda la colección a una nueva variable.

La principal ventaja de los tipos estáticos es la eficiencia: como sabemos el tamaño de la variable cuando la declaramos, Java no necesita preocuparse por comprobar el tamaño cada vez que usamos la variable, por lo que los programas son más rápidos. Sin embargo, el principal inconveniente de tratar con estos tipos es que son demasiado rígidos, y si necesitamos aumentar el tamaño previamente establecido, el código puede volverse bastante complicado e ineficiente.

2. ESTRUCTURAS DE DATOS

Una estructura de datos es una organización particular de una colección de elementos

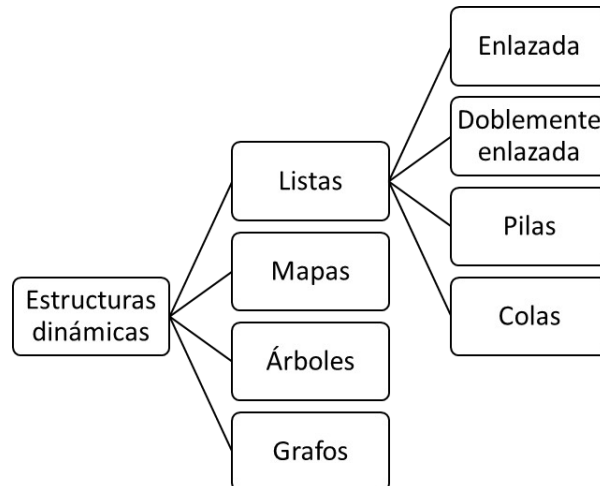
datos con unas propiedades que la definen. Así, por ejemplo, una lista ordenada de números enteros almacenados en un array es un ejemplo sencillo de estructura de datos. La complejidad de la información que maneja una aplicación y los algoritmos que se emplean, hoy en día requieren, más que nunca, la existencia de las estructuras de datos. Las estructuras pueden ser estáticas o dinámicas, siendo la mayoría de las que se usan de tipo dinámico.

Será necesario que la elección de una estructura de datos adecuada requiera también la posibilidad de poder realizar operaciones concretas sobre dichas estructuras. La **elección** de la estructura de datos adecuada al problema a resolver es una cuestión vital, que redundará en una mayor eficiencia del programa y, sobre todo, en una mejor resolución del problema en cuestión. Una elección inadecuada de la estructura de datos puede conducir a programas lentos, largos y poco eficientes; e incluso a la imposibilidad de resolver el problema.

Son varias las **razones** que las hacen **importantes** para el **desarrollo** de aplicaciones, entre otras las siguientes:

1. **Eficiencia en el manejo de datos:** diseñadas para manejar grandes cantidades de datos de manera rápida y eficiente. Las aplicaciones suelen manejar gran cantidad de información: usuarios, transacciones, pedidos, comentarios, etcétera...
2. **Flexibilidad en el almacenamiento y manipulación de datos:** ofrecen una gran variedad de opciones para manipular los datos de manera que se ajusten a las necesidades específicas de una aplicación. Por ejemplo, las listas enlazadas permiten añadir, eliminar o buscar elementos de manera eficiente, mientras que los árboles binarios pueden ser usados para realizar búsquedas y ordenaciones muy rápidas..
3. **Capacidad para solucionar problemas complejos:** ofrecen el soporte para resolver problemas complejos mediante los algoritmos. Por ejemplo, los grafos son ideales para representar y resolver problemas que involucren relaciones entre objetos o entidades, como en una red social o la navegación en un sistema GPS. Las estructuras de datos avanzadas pueden ser la clave para solucionar problemas difíciles en el desarrollo de aplicaciones.
4. **Escalabilidad:** pueden manejar grandes volúmenes de datos sin disminuir su eficiencia aumentando los costes computacionales de forma lineal y proporcional. Con el aumento del volumen de datos en una aplicación web, es necesario que las estructuras de datos puedan manejar este crecimiento sin comprometer la velocidad y eficiencia de la aplicación.

A continuación, podemos observar una clasificación **detallada**, de las estructuras concretas a estudiar (desde un punto de vista teórico) en esta unidad.



Tras su análisis conceptual trabajaremos en Java con las listas y los mapas, dejando de lado los árboles y los grafos, dado que Java no proporciona clases preparadas para trabajar con ellos y dada la complejidad de su implementación manual.

2.1 Fundamentos de estructuras de datos

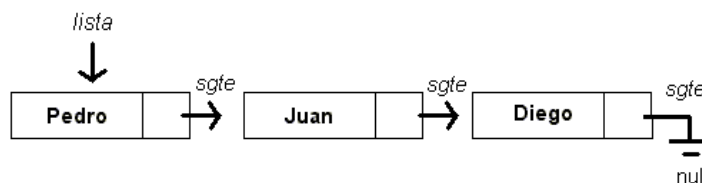
En este apartado, vamos a realizar una exposición teórica de las estructuras de datos desde un punto de vista abstracto y generalista, el cual nos servirá para entenderlas un poco mejor antes de pasar a utilizarlas en nuestras aplicaciones a través del lenguaje Java. Es importante destacar que vamos a realizar un acercamiento conceptual a dichas estructuras, que nada tiene que ver con Java. Son conceptos extrapolables y aplicables a cualquier lenguaje que los soporte.

1. Listas

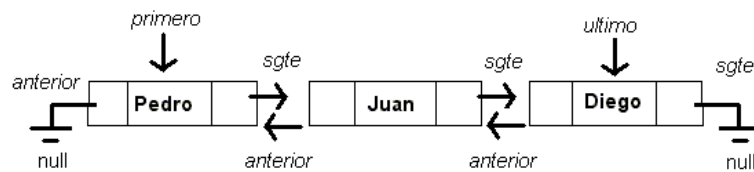
Las listas están formadas por un conjunto de elementos, llamados nodos, normalmente homogéneos, enlazados entre sí y que se recorren de forma secuencial; por lo que, para acceder a un elemento en la posición N, es necesario recorrer los N-1 elementos anteriores a él. Aspectos como por ejemplo si la lista está ordenada o no, pueden cambiar la forma de trabajar con ellas. A efectos prácticos serían una evolución de los arrays.

Existen 4 subtipos básicos de listas:

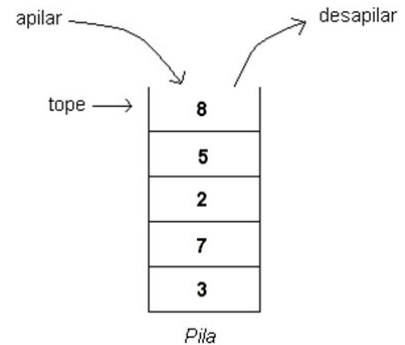
- **Lista enlazada** (acceso secuencial en 1 dirección)



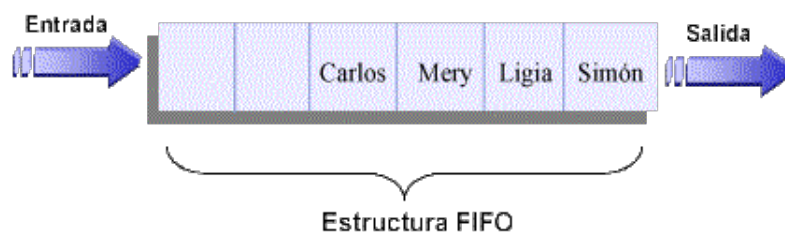
- **Lista doblemente enlazada** (acceso secuencial en 2 direcciones)
-



- **Pila:** particularidad de la lista enlazada donde los elementos se insertan y extraen desde el mismo extremo. Suelen implementar LIFO (Last In First Out). Algunas de sus aplicaciones son: almacenamiento en memoria de llamadas recursivas a funciones, la función “deshacer / rehacer”, mantenimiento de un listado de “archivos recientes”, la realización de operaciones en las calculadoras, etcétera...



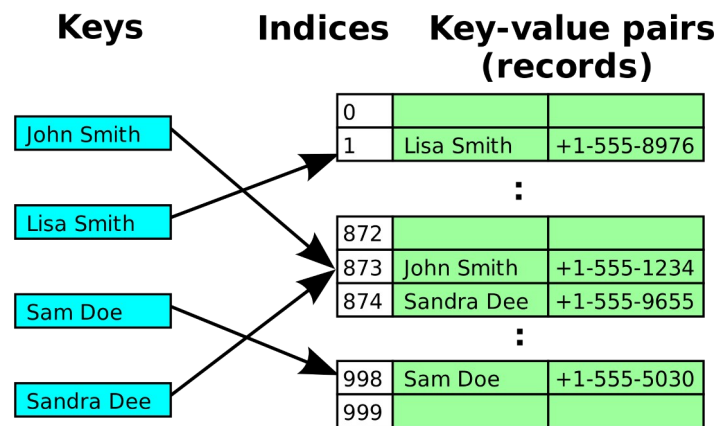
- **Cola:** tipo de lista con la particularidad de que sólo permite insertar elementos por un lado y extraerlos por el otro. Generalmente implementan FIFO (First In First Out). Algunas de sus aplicaciones son: gestionar prioridades (las listas de procesos a ejecutar por la CPU, la cola de la caja del supermercado, la cola de impresión...), etc...



2. Mapas o tablas “hash”

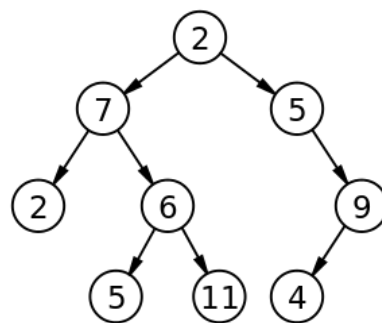
En estas estructuras los elementos se referencian por una clave en vez de por un índice numérico (como en los arrays). De esa forma, lo que se almacena en una tabla hash son pares de claves y valores (o registros) asociados, por lo que conociendo la clave podemos acceder directamente al valor asociado sin necesidad de recorrer la estructura, tal como lo hacemos cuando buscamos la definición de una palabra en un diccionario: buscamos una clave (la palabra) y luego obtenemos su definición. Esto se hace efectivo mediante una “función hash” que calcula la correspondencia de la clave a la ubicación del valor.

Por ejemplo, una tabla hash donde la clave sea el DNI y el valor asociado una estructura llamada “Persona” con los datos personales de una persona. O bien una donde, sabiendo el nombre, podamos acceder al registro del teléfono de una persona...para entender mejor su objetivo, piensa, ¿cómo almacenarías esto en un array normal?



3. Árboles

Esta estructura, ampliamente usada y que imita la forma de un árbol, representa un conjunto de nodos donde cada uno está enlazado con otros nodos llamados hijos formando una jerarquía no cíclica. Si cada nodo sólo puede tener como máximo 2 hijos (izquierdo y derecho) se llama árbol binario. Veamos un ejemplo gráfico del árbol binario:



Terminología básica:

- **Raíz:** El nodo superior e inicial de un árbol.
- **Hijo:** Un nodo conectado directamente con otro cuando se aleja de la raíz.
- **Padre:** La noción inversa de hijo.
- **Hermanos:** Un conjunto de nodos con el mismo padre.
- **Hoja:** Un nodo sin hijos.
- **Camino:** Una secuencia de nodos y brazos conectados con un nodo descendiente.

Si bien poseen las mismas operaciones que las listas (insertar, eliminar y buscar), la forma de recorrerlos puede ser diferente, siendo la manera más efectiva de hacer estos recorridos la recursiva. Veamos el orden de recorrido de los nodos de las formas más destacadas:

- **Preorden:** raíz, izquierdo, derecho.
- **Inorden:** izquierdo, raíz, derecho.
- **Postorden:** izquierdo, derecho, raíz.

Los árboles se emplean en multitud de problemas, como por ejemplo, en los algoritmos de encaminamiento (ruta óptima en GPS).

Hay un caso particular de árbol que se llama **“heap”** o **montículo**. Es una estructura de datos muy relevante aunque sólo la vamos a mencionar brevemente. Se caracteriza por ser un árbol binario donde cada nodo tiene una ordenación siempre igual (o bien cada nodo padre tiene un valor mayor que el de cualquiera de sus nodos hijos o al revés, el valor del nodo padre es siempre menor al de sus nodos hijos). Éstos tienen multitud de aplicaciones, como por ejemplo la implementación de colas de prioridad, la gestión de memoria dinámica, etc...

4. Grafos

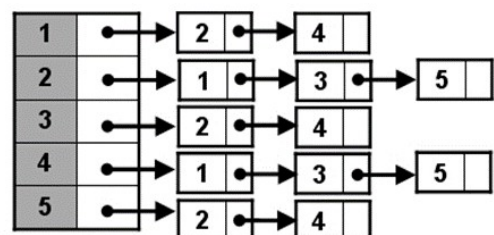
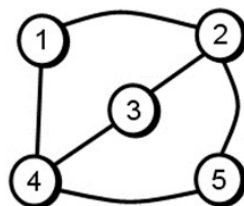
Estructuras matemáticas similares a los árboles, pero no jerárquicas, formadas por un conjunto de nodos (vértices) y aristas (uniones entre 2 vértices) que pueden tener una distancia (un peso o un coste) y, además, pueden ser dirigidos o no. Si sus aristas tienen peso se les llama ponderados.

Las operaciones básicas a realizar sobre grafos son añadir y eliminar nodos y arcos. También podemos recorrer un grafo, de forma similar a un árbol, aunque mediante el sistema de arcos. En este caso, es habitual calcular el coste (suma de pesos) de un recorrido por el grafo para ir de un nodo a otro.

Infinidad de algoritmos emplean los grafos como estructura de datos. Podemos citar como ejemplos el algoritmo de Kruskal, el de Dijkstra, el de Ford-Fulkerson o el de Bellman-Ford. Los algoritmos de enrutamiento dinámico, empleados por los routers en el nivel IP de las redes de comunicaciones, emplean grafos para su funcionamiento y algoritmos como algunos de los mencionados.

Se les puede representar mediante matrices de adyacencia o listas de adyacencia. Veamos un ejemplo de grafo no dirigido representado en ambas formas, con matriz y con lista:

M	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	1
3	0	1	0	1	0
4	1	0	1	0	1
5	0	1	0	1	0

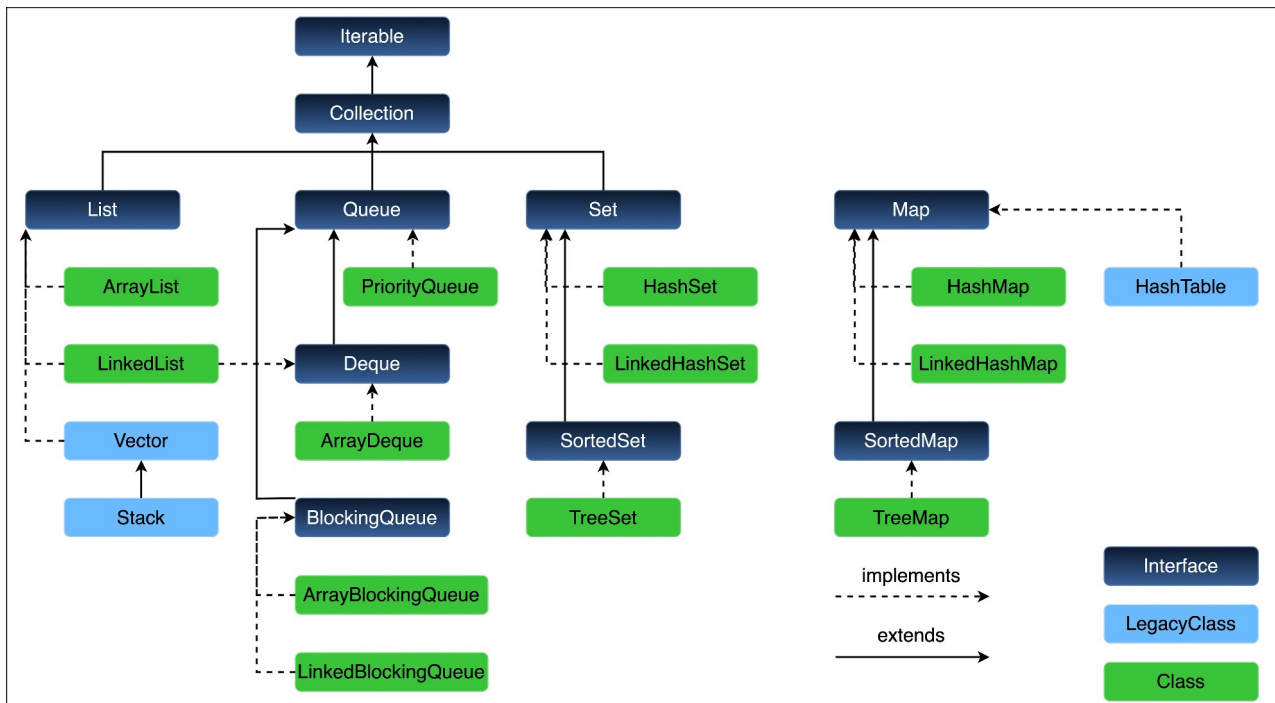


2.2 Estructuras de datos en Java: Colecciones

Una vez hemos conocido las estructuras de datos más relevantes desde un punto de vista conceptual, vamos a ver qué nos ofrece Java para poder representarlas. En Java (y C#), a las estructuras de datos dinámicas se las conoce como “Colecciones”.

Es importante empezar recordando que el framework de Colecciones en Java no proporciona clases para modelar árboles o grafos, y se deja a la responsabilidad del programador su implementación manual y concreta mediante clases y otras estructuras. Dentro de las colecciones existen unas interfaces principales: **List**, **Queue**, **Set** y **Map**. Cuando se crea una clase que implementa alguna de estas interfaces, está obligada a tener los métodos que indica dicha interfaz.

Veamos en la siguiente imagen la jerarquía del framework “Collection”:



De todas ellas, las estructuras más relevantes son:

- **ArrayList** (lista de acceso directo por índice tipo array)
- **LinkedList** (lista enlazada y doblemente enlazada)
- **Queue** (cola usando **LinkedList**) y **Deque** (double ended queue)
- **Stack** (pila)
- **HashMap** (tabla Hash)
- **Set** (conjunto)

Veamos sus detalles:

Listas (List)

Las colecciones de tipo **List** son las más “parecidas” a los arrays estáticos y permiten instanciar objetos de clases como **ArrayList** y **LinkedList**. Son las más utilizadas, aunque dependiendo de la finalidad de la colección que necesitemos tenemos que valorar el uso de otros tipos de estructuras dinámicas como son los conjuntos (**Set**) y los mapas (**Map**).

Conjuntos (Set)

Los conjuntos en Java, al igual que los conjuntos en matemáticas, no permiten contener elementos duplicados. Estas clases implementan la interface **Set**:

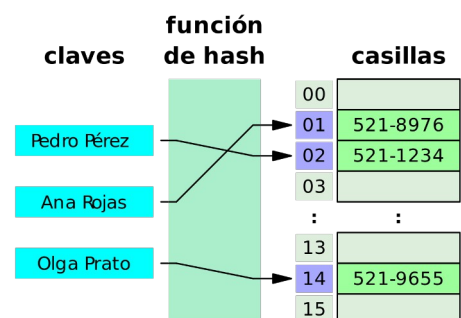
- **HashSet** : Almacena los elementos en una tabla hash sin mantener ningún orden.
- **TreeSet** : Almacena los elementos ordenándolos en función de sus valores. Necesita que los elementos implementen la interface `Comparable`
- **LinkedHashSet** : Almacena los elementos en función de su orden de inserción.

Mapas (Map)

Corresponden al concepto de diccionario o pares clave-valor, habitualmente en forma de tabla hash. Son colecciones que contienen asociaciones *clave:valor*, esto quiere decir que al añadir un elemento (valor) a la colección debemos añadir una clave para identificarlo. Por ejemplo: `key` → dni, `value` → persona. Estas estructuras implementan la interface **Map**.

Para recuperar los elementos añadidos, debemos utilizar la clave de cada elemento.

Key	Value
714435667R	Persona (nombre = "Juan", apellidos = "López")
985543264X	Persona (nombre = "Maria", apellidos = "Martinez")
98773255L	Persona (nombre = "Javier", apellidos = "López")



En el ejemplo superior, almacenamos personas como elementos, y los índices (o claves) para acceder a ellos son el DNI.

En Java, las clases más relevantes que implementan este concepto son:

- **HashMap**: almacena las claves en una tabla hash sin orden (la más habitual).
 - **TreeMap** : almacena las claves ordenándolas en función de sus valores.
 - **LinkedHashMap** : almacena las claves en función de su orden de inserción.
-

2.3 Listas: la clase LinkedList

Esta clase implementa el concepto general de una lista, mediante el modelo de la doblemente enlazada. La usamos cuando lo que queremos es insertar o borrar en una posición determinada y nos prima la eficiencia en esa operación. Es una especie de **ArrayList** que no tiene definido una posición de inicio ni de fin, emplea básicamente lo mismos métodos y de la misma forma que **ArrayList** ya que son clases que implementan la interfaz **List** (orden superior a ambas). En un **ArrayList** los elementos están en posiciones de memoria contiguas, mientras que en un **LinkedList** no necesariamente.

A continuación, 2 ejemplos para instanciar una **LinkedList** (es preferible el 2º):

```
LinkedList<String> lista1 = new LinkedList<String>();  
List<String> lista1 = new LinkedList<String>();
```

La razón de esto es que, al declarar la lista como una instancia de la interfaz **List**, el código es más flexible y extensible, haciendo uso del polimorfismo. Si en algún momento se desea cambiar la implementación de la lista a otra clase (por ejemplo, de **LinkedList** a **ArrayList**), se puede hacer simplemente cambiando la línea de código donde se instancia la lista, sin necesidad de modificar el resto del código, ya que los métodos usados en una y otra son los mismos (vienen definidos en la interfaz **List**).

Además, se promueve el encapsulamiento y la abstracción, ya que se utiliza la interfaz como una "caja negra" que oculta los detalles de implementación de la lista y se enfoca en su comportamiento y funcionalidad.

Mediante el método '**add**' añadimos nodos al final de la lista que hemos creado en este ejemplo:

```
lista1.add("Juan");  
lista1.add("Luis");  
lista1.add("Carlos");
```

El método '**add**' de la clase **LinkedList** se encuentra sobrecargado existiendo un segundo método '**add**' con 2 parámetros que recibe: en el primer parámetro la posición donde se debe insertar el nodo y como segundo parámetro el dato a almacenar:

```
lista1.add(1, "ana");
```

Para eliminar un nodo de la lista debemos llamar al método '**remove**' y pasar la posición del nodo a eliminar:

```
lista1.remove(0);
```

También podemos eliminar los nodos que coinciden con cierta información:

```
lista1.remove("Carlos");
```

La cantidad de nodos nos lo suministra el método 'size':

```
System.out.println("Cantidad elementos en lista:" + lista1.size());
```

El siguiente método estático 'imprimir' muestra todos los elementos de la lista. Recibe la lista por parámetro y mediante un foreach recorre la colección mostrando sus elementos:

```
public static void imprimir(LinkedList<String> lista) {  
    for (String elemento : lista)  
        System.out.print(elemento + "-");  
    System.out.println();  
}
```

Para conocer si la lista almacena cierto valor disponemos del método 'contains':

```
if (lista1.contains("Ana"))  
    System.out.println("El nombre 'Ana' si esta en la lista");  
else  
    System.out.println("El nombre 'Ana' no esta en la lista");
```

Para recuperar el dato de un nodo sin eliminarlo podemos hacer uso del método 'get':

```
System.out.println("El 2º elemento es:" + lista1.get(1));
```

Eliminamos todos los nodos de la lista mediante el método 'clear':

```
lista1.clear();
```

Podemos consultar si la lista se encuentra vacía mediante el método 'isEmpty':

```
if (lista1.isEmpty())  
    System.out.println("La lista se encuentra vacía");
```

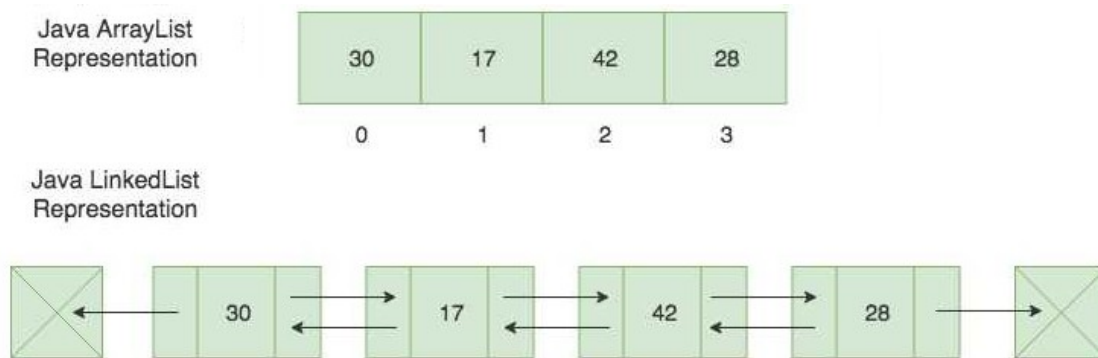
La clase `LinkedList` también nos permite tratar la lista como si fuera doblemente enlazada gracias a determinados métodos de la interfaz `Iterator`, como son: `.next()` y `.previous()`

¿Cuándo usar `LinkedList` y cuando `ArrayList`? Si se requiere añadir y borrar datos con mucha frecuencia, `LinkedList` será la mejor opción. En cambio, si hay pocas operaciones de inserción y borrado, y muchas de búsqueda y consulta, entonces mejor usar `ArrayList`.

Reflexiona sobre curiosidades como:

- `ArrayList<ArrayList<Integer>> matriz = new ArrayList<ArrayList<Integer>>(n);`
 - `ArrayList<Integer>[] al = new ArrayList[n];`
-

Veamos gráficamente cómo se representan en memoria ambas estructuras de datos:



2.4 Colas: la interfaz Queue

Una cola FIFO, en Java, se puede implementar usando la interfaz “Queue” y la clase “LinkedList”. Así podremos administrar una lista con la **funcionalidad** de una cola (dado que, como vimos al principio de la unidad, una cola es un tipo particular de lista).

```
Queue<String> cola1 = new LinkedList<>();
```

La interfaz Queue es la que declara los métodos principales para trabajar una cola. Añadimos elementos al final de la cola mediante el método 'add':

```
cola1.add("juan");  
cola1.add("ana");
```

Para conocer la cantidad disponemos del método **size()**:

```
System.out.println("Cantidad elementos en la cola:" + cola1.size());
```

Para extraer el nodo del principio de la cola lo hacemos mediante el método 'poll':

```
System.out.println("Extraemos elemento de la cola:" + cola1.poll());
```

Si queremos conocer el primer elemento de la cola sin extraerlo usamos el método 'peek':

```
System.out.println("Consulta 1er elemento:" + cola1.peek());
```

Para conocer si la cola se encuentra vacía llamamos al método 'isEmpty':

```
while (!cola1.isEmpty())  
    System.out.print(cola1.poll() + "-");
```

Podemos utilizar un `foreach` para recorrer todos los elementos de la colección de tipo **Queue** con la siguiente sintaxis (no se eliminan los elementos de la cola):

```
for (Integer elemento : cola2)
    System.out.print(elemento + "-");
```

Para eliminar todos los elementos de una pila empleamos el método `clear`:

```
cola2.clear();
```

Colas de prioridad

En Java también existe la **clase** “`PriorityQueue`”, que permite instanciar directamente una cola de prioridad. **Ojo**: las colas de prioridad ya no respetan el FIFO estrictamente. Veamos un ejemplo:

```
PriorityQueue<Integer> colaPrioridad = new PriorityQueue<Integer>();

colaPrioridad.add(70);
colaPrioridad.add(120);
colaPrioridad.add(6);

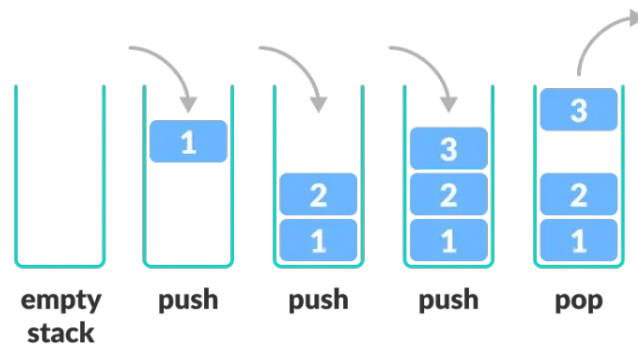
System.out.println("Mostrando cola de prioridades de enteros:");
while(!colaPrioridad.isEmpty())
    System.out.print(colaPrioridad.poll() + "-");
```

Cuando comenzamos a recuperar los elementos de la cola con prioridad podemos comprobar que el primero en ser procesado de la cola será el que tiene el valor 6, luego el 70 y finalmente el 120; en vez de procesarse en el orden en que fueron encolados. Esto muestra cómo una `PriorityQueue` en Java puede ordenar automáticamente los elementos según su valor natural (en este caso, de menor a mayor) sin necesidad de un comparador personalizado.

2.5 Pilas: la clase Stack

Una pila (stack) es una estructura que funciona como una lista LIFO (Last In First Out). El último elemento añadido será el primero en salir. Se puede entender como una pila de platos, se van colocando uno encima de otro y se van cogiendo desde arriba hacia abajo, o de libros o de cualquier objeto. Tiene 5 métodos principales, además de todos los que posee la interfaz “`List`”, ya que la implementa internamente:

- **push(Elemento)**: apilar; añade un elemento a la parte “superior” de la pila.
 - **pop()**: desapilar; extrae el elemento de la cima de la pila y lo elimina de ella.
 - **empty()**: indica si la pila está vacía.
 - **peek()**: consulta el elemento de la cima pero sin desapilarlo.
 - **search(Object)**: indica la posición (int) del elemento “Object” en la pila.
-



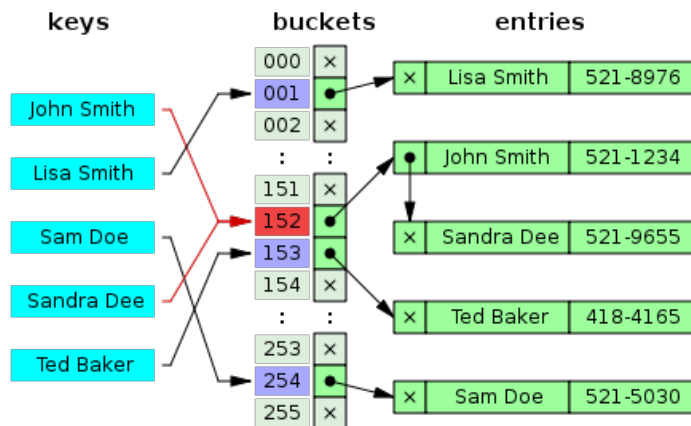
Para poder usar una pila debemos empezar instanciando un objeto (2 ejemplos diferentes):

```
Stack<String> pilaStrings = new Stack<>();  
Stack<Libro> pilaLibros = new Stack<>();
```

Ejemplo de uso de una pila:

```
import java.util.Stack;  
  
Stack<String> pila1 = new Stack<String>();  
  
//apilar 3 elementos (de tipo String)  
pila1.push("Programación");  
pila1.push("Bases de datos");  
pila1.push("Otras");  
  
//mostrar por consola (no recomendable...)  
for(String cadena : pila1){  
    System.out.println(cadena);  
}  
  
//elimina y devuelve el elemento que está en la cima  
String desapilado = pila1.pop();  
System.out.println(desapilado);  
  
//devuelve el elemento que está en la cima sin borrarlo  
String ultimo = pila1.peek();  
System.out.println(ultimo);
```

2.6 Mapas: la clase HashMap



De todos los mapas disponibles en Java, la clase “HashMap” es una de las más utilizadas. En la teoría se les conoce también como tablas hash. Ejemplos gráfico:

Para crear un objeto de la clase HashMap podemos hacerlo con la siguiente sintaxis:

```
Map<String, Persona> registro = new HashMap<String, Persona>();  
HashMap<String, Persona> registro = new HashMap<String, Persona>();
```

La clase HashMap utiliza datos genéricos tanto para la clave como para el valor, en el ejemplo anterior la clave es String y el valor almacenado es un objeto de tipo Persona. Para hacer uso de esa clase se requiere incluir: `import java.util.HashMap;`

Veamos los métodos más destacados. Mediante el método **put** agregamos un elemento al HashMap. Desconocemos donde lo va a ubicar físicamente ya que internamente realiza un cálculo (función de hashing) para obtener el lugar donde situarlo:

```
tabla1.put("rojo", "red");  
tabla1.put("verde", "green");  
tabla1.put("azul", "blue");
```

Si queremos recorrer todas las **claves** del mapa:

```
for (String clave : tabla1.keySet())  
    System.out.print(clave + "-");
```

Para recuperar un valor para una determinada clave llamamos al método '**get**' y le pasamos la clave a buscar, si dicha clave no existe en el mapa nos devuelve el valor 'null':

```
System.out.println("A 'rojo' corresponde:" + tabla1.get("rojo"));
```

Para mostrar todos los **valores** del mapa lo recorreremos mediante un **for**:

```
for (String valor : tabla1.values())  
    System.out.print(valor + "-");
```

Si queremos verificar si una determinada clave existe en el mapa lo hacemos mediante el método '**containsKey**':

```
if (tabla1.containsKey("negro"))  
    System.out.println("No existe la clave 'negro'");
```

Una variante del método '**get**' es '**getOrDefault**' que nos retorna el segundo parámetro si no encuentra la clave en el mapa:

```
System.out.println("La traducción de 'marron' es:" +  
    tabla1.getOrDefault("marrón", "No existe la clave 'marrón'"));
```

Para eliminar un elemento de la colección debemos hacer uso del método '**remove**' pasándole una clave del mapa:

```
tabla1.remove("rojo");
```

Para imprimir el mapa completo en la Consola podemos hacer uso del método '**println**':

```
System.out.println(tabla1);
```

2.7 Conjuntos en Java: la interfaz Set

Los conjuntos son un tipo de estructura de datos que, si bien estructuralmente no se diferencian de una lista, funcionalmente sí ya que NO permiten elementos duplicados. En Java existe una interfaz que permite implementar conjuntos y, por ese motivo, y por el hecho de que son una estructura relevante, vamos a mencionarla. Un objeto **Set** es un objeto **Collection** que contiene elementos únicos (no duplicados). El framework de colecciones de Java contiene diferentes implementaciones de Set incluyendo **HashSet** y **TreeSet**. El primero almacena los elementos en una tabla **hash** y el segundo en un **árbol**.

En el siguiente ejemplo, primero creamos un HashSet y agregamos 2 elementos. Luego creamos otro HashSet y agregamos 2 elementos más. Esto luego se agrega como una colección al primer conjunto. En el ejemplo de eliminación, primero eliminamos un elemento del primer conjunto y luego eliminamos todos los elementos del segundo conjunto.

```
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

public class SetExample {

    public static void main(String[] args) {

        Set<String> conjuntoString = new HashSet<String>();
        conjuntoString.add("Java");
        conjuntoString.add("Javascript");
        System.out.println(conjuntoString);

        Set<String> string2 = new HashSet<String>();
        string2.add("C");
        string2.add("C++");

        conjuntoString.addAll(string2);
        System.out.println("Elementos tras addAll");
        System.out.println(conjuntoString);

        conjuntoString.remove("Javascript");
        System.out.println("Elementos tras borrado");
        System.out.println(conjuntoString);

        conjuntoString.removeAll(string2);
        System.out.println("Elementos tras borrar todos");
        System.out.println(conjuntoString);
    }
}
```

2.7.1 Conjuntos ordenados

El framework de colecciones de Java incluye también la interfaz **SortedSet** (que extiende a **Set**) para conjuntos que mantengan sus elementos ordenados, ya sea en el orden natural de los elementos o por un orden especificado por un objeto **Comparator**. La clase **TreeSet**

implementa a **SortedSet**.

Veamos un ejemplo de implementación. Hemos declarado una matriz de enteros y luego agregamos los elementos al HashSet usando el bucle for. En la salida se puede ver que el elemento duplicado "12" no está incluido, que es la característica establecida. Podemos ordenar los elementos de HashSet usando TreeSet:

```
import java.util.*;

public class SetSort {

    public static void main(String[] args) {

        Set<Integer> numbers = new HashSet<Integer>();
        int[] a = {34,78,12,67,45,23,12,89};
        for(int i=0;i<a.length;i++) {
            numbers.add(a[i]);
        }
        System.out.println("Elements in HashSet");
        System.out.println(numbers);

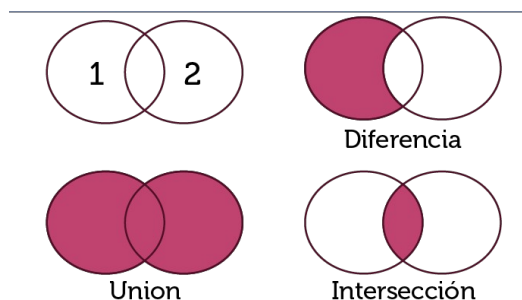
        Set<Integer> treeset = new TreeSet<Integer>(numbers);
        System.out.println("Elements in treeset after sort");
        System.out.println(treeset);
    }
}
```

2.7.2 Aplicaciones a la teoría de conjuntos: unión, intersección y diferencia

Los conjuntos tienen infinidad de aplicaciones, en principio todas aquellas donde en la propia matemática se emplean. Sin embargo, un gran campo de aplicación es la teoría de conjuntos; ampliamente utilizada en probabilidad y otros campos.

Para entender lo que vamos a realizar, y antes de la implementación, haremos una pequeña introducción. La teoría de conjuntos es una rama de la lógica matemática que estudia las propiedades y relaciones de los conjuntos: **agrupaciones** abstractas de objetos, consideradas como objetos en sí mismas. La teoría de conjuntos es lo suficientemente rica como para construir el resto de objetos y estructuras de interés en matemáticas: números, funciones, figuras geométricas, etc.

Existen unas operaciones básicas que permiten manipular los conjuntos y sus elementos, similares a las operaciones aritméticas, constituyendo el álgebra de conjuntos, estas son: la unión, la intersección y la diferencia. Existen otras pero nos centraremos en estas últimas. Veamos un ejemplo gráfico que las ilustra, se llaman diagramas de Venn:



Veamos, en Java, como se puede emplear la interfaz Set y la clase HashSet para implementar la lógica de conjuntos anterior:

```
import java.util.*;

public class SetOperations {

    public static void main(String[] args) {

        Set<Integer> set1 = new HashSet<Integer>();
        set1.add(3);
        set1.add(7);
        set1.add(5);
        set1.add(1);
        System.out.println("Set 1 : " + set1);

        Set<Integer> set2 = new HashSet<Integer>();
        set2.add(3);
        set2.add(4);
        set2.add(9);
        set2.add(5);

        System.out.println("Set 2 : " + set2);

        Set<Integer> a = new HashSet<Integer>(set1);
        a.addAll(set2);
        System.out.println("Union de 1 y 2 : " + a);

        Set<Integer> b = new HashSet<Integer>(set1);
        b.retainAll(set2);
        System.out.println("Intersección de 1 y 2 : " + b);

        Set<Integer> c = new HashSet<Integer>(set1);
        c.removeAll(set2);
        System.out.println("Diferencia entre 1 y 2 : " + c);
    }
}
```

3. ANÁLISIS Y DISEÑO CON ESTRUCTURAS DE DATOS

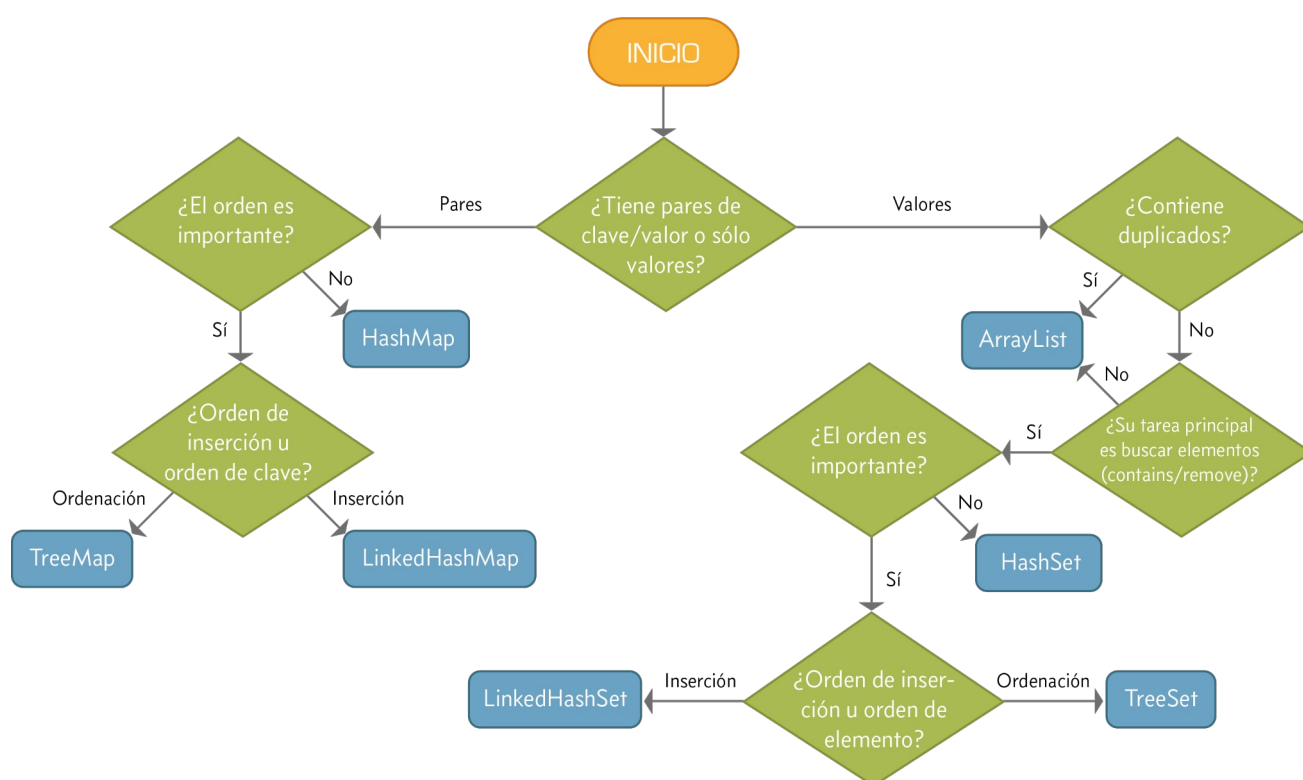
La elección de las estructuras de datos a emplear en la aplicación es absolutamente fundamental y tiene un gran impacto en el rendimiento de la misma, incluso en la funcionalidad. En la fase de diseño técnico debemos tomar decisiones acerca de qué estructuras emplear y porqué, con el objetivo de lograr la eficiencia y eficacia de la aplicación.

Algunos aspectos a tener presentes son: tiempo de ejecución requerido por la aplicación, espacio de almacenamiento usado (y su tipo: estático o dinámico), cantidad de operaciones de escritura a realizar, cantidad de operaciones de lectura a realizar, tipología de los datos a almacenar y características o particularidades de los mismos así como la relación entre ellos (duplicados....).

¿Qué estructura de datos elegir?

El siguiente diagrama de flujo ofrece una forma de seleccionar el tipo de colección a emplear, en Java, según unos factores o criterios:

Diagrama de decisión para uso de colecciones Java



Recomendaciones para elegir estructura de datos

1. Colección de elementos simple y de tamaño fijo: array.
2. Colección dinámica y de tamaño variable que admita acceso aleatorio: ArrayList.
3. Colección dinámica y de tamaño variable que admita una inserción y eliminación rápidas: LinkedList.
4. Colección ordenada sin elementos duplicados: TreeSet o SortedSet.
5. Colección sin elementos duplicados y con operaciones rápidas de inserción, eliminación y búsqueda: HashSet.
6. Colección sin elementos duplicados y que mantenga el orden de inserción: LinkedHashSet.
7. Colección de pares clave-valor con operaciones rápidas de inserción, eliminación y búsqueda: HashMap.
8. Colección de pares clave-valor con claves ordenadas: TreeMap o SortedMap.
9. Colección de pares clave-valor que mantenga el orden de inserción: LinkedHashMap.
10. Cola FIFO: LinkedList (con Queue) o un ArrayDeque.
11. Cola de prioridad con acceso eficiente al elemento de prioridad más pequeño o más alto: PriorityQueue.
12. Cola de dos extremos (Deque) con operaciones rápidas de inserción y eliminación en ambos extremos: LinkedList (con Deque) o ArrayDeque.

4. COMPLEJIDAD COMPUTACIONAL

Antes de empezar el bloque de algoritmos, es importante que estudiemos un área necesaria: la Teoría de la Complejidad Computacional. En ella se estudian los recursos requeridos durante el cálculo para resolver un problema. Así, un cálculo difícil requerirá más recursos que uno de menor dificultad. Los recursos comúnmente estudiados son: el tiempo (número de pasos de ejecución de un algoritmo para resolver un problema) y el espacio (cantidad de memoria utilizada para resolver un problema).

Un algoritmo que resuelve un problema pero tarda mucho tiempo en hacerlo o utiliza demasiada memoria, no será de utilidad.

Orden de complejidad

Para analizar el rendimiento de un algoritmo de un modo sencillo, se usa la notación Big O, la cual es una notación matemática para expresar cuanto tarda un algoritmo en ejecutarse atendiendo sólo a grandes rasgos su eficiencia, generalmente centrándonos en el caso promedio o peor. En concreto se usa para expresar de forma abstracta la velocidad de procesamiento de un algoritmo, atendiendo a cómo aumentaría su tiempo de ejecución en función de un aumento significativo del tamaño de los datos de entrada.

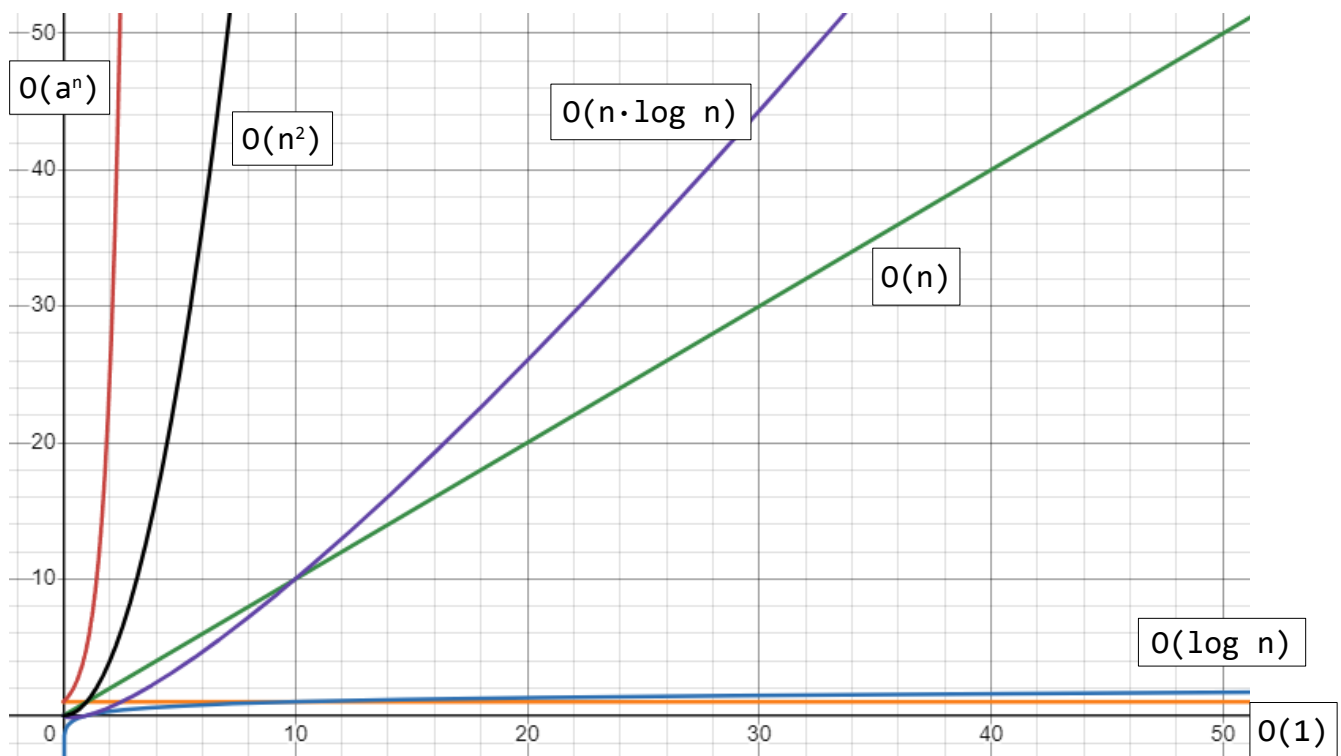
La O viene de clasificar el ORDEN de magnitud de la función a analizar, resumiéndose en que nos sirve para determinar cómo un algoritmo escala en complejidad tanto en tiempo como en espacio necesario, en función de su tamaño de entrada. Podemos resumir la nota dada a unos pocos valores, ya que descartamos las constantes y sólo atendemos a unos pocos casos según el orden de magnitud de crecimiento, y sin importar por ejemplo el detalle de la inclinación de cada curva y sólo la forma de comportamiento de crecimiento. En resumen, nos podemos quedar con que tenemos:

- $O(1)$ - tiempo **constante**: es el mejor resultado, y quiere decir que el tiempo de ejecución no varía conforme aumenta el tamaño de los datos de entrada, y la respuesta siempre tarda lo mismo sin importar la magnitud de entrada.
 - $O(\log n)$ - tiempo **logarítmico**: una forma de crecimiento que crece al inicio pero tiende a estabilizarse conforme aumentan el tamaño de entrada, por lo que es una buena nota para un algoritmo ya que no tiende a resentirse.
 - $O(n)$ - tiempo **lineal**: el crecimiento es lineal en tanto el tiempo de ejecución es cada vez mayor de modo proporcional a cómo se incrementa el tamaño de la entrada. Por lo que si tenemos el doble de elementos de entrada, tardará el doble, aunque despreciamos realmente la pendiente de la misma y sólo nos quedamos con que aumenta de forma lineal.
 - $O(n \cdot \log n)$ - tiempo **lineal-logarítmico** (mal llamado “casi lineal”): es un coste aceptable, en términos generales. El algoritmo de Quicksort se engloba en este coste computacional.
 - $O(n^2)$ - tiempo **cuadrático** (o polinómico): el crecimiento es de forma potencial por lo que será un algoritmo a evitar ya que para valores pequeños de entrada el tiempo será
-

asumible, pero conforme aumente el tamaño de los datos de entrada el tiempo tenderá a ser muy elevado y es probable que el procesador se quede inoperativo.

- $O(a^n)$ - tiempo **exponencial**, por ejemplo $O(2^n)$: el crecimiento es de forma exponencial, por lo que el coste computacional y espacial se dispara con tamaños de entrada pequeños. Debe evitarse.
- $O(n!)$ - tiempo **factorial**: el crecimiento es factorial, por lo que rápidamente tiende a valores imposibles de tratar, en lo que sería una recta vertical. Debe evitarse a toda costa.

Para entender mejor esto, adjuntamos una gráfica comparativa:



En el eje X (valor de n) se representaría el tamaño de la estructura de datos a procesar, y en el eje Y se representaría el tiempo necesario. Podemos observar claramente las diferencias significativas entre los distintos órdenes de complejidad.

A nivel práctico, por ejemplo, si consultamos la documentación de MSDN para la función RemoveAt de una SortedList nos dice qué es una operación $O(n)$:

Remarks

The index sequence is based on the sort sequence. When an element is added, the index sequence adjusts accordingly. When an element is removed, the indexing also adjusts as elements are added or removed from the [SortedList](#) object.

In collections of contiguous elements, such as lists, the elements that follow the collection is indexed, the indexes of the elements that are moved are also updated. Conceptually grouped into buckets, such as a hash table.

This method is an $O(n)$ operation, where n is [Count](#).

Y si consultamos el API de Java, en la clase Arrays, el método “sort” nos indica en su descripción:

*The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. **This algorithm offers $O(n \log(n))$ performance on many data sets that cause other quicksorts to degrade to quadratic performance, and is typically faster than traditional (one-pivot) Quicksort implementations.***

¿Qué nos indica esa descripción acerca de la complejidad del método “sort”?... Veamos algunos **ejemplos** en Java:

- Coste lineal $O(n)$: código para búsqueda secuencial (recorrer un vector y comprobar cada posición).
- Coste logarítmico $O(\log n)$: ver ejemplo de “búsqueda binaria”.
- Coste cuadrático $O(n^2)$:

```
void buscarRepetidos(int[] vector){
    for(int i=0;i<vector.size;i++)
        for(int j=0;j<vector.size;j++)
            if(i!=j && vector[i] == vector[j])
                System.out.println("Elemento " + vector[i] + "repetido!");
}
```

- Coste exponencial $O(2^n)$: calcular el número de fibonacci.

```
int fibonacci(int numero){
    if(numero <= 1)
        return numero;
    else
        return fibonacci(numero - 2) + fibonacci(numero - 1);
}
```

5. ALGORITMOS SOBRE ESTRUCTURAS DE DATOS

Los algoritmos son la piedra angular sobre la cual se construyen soluciones eficientes y efectivas para una amplia gama de problemas. En este apartado vamos a explorar algoritmos habituales que abarcan desde técnicas de ordenación hasta estrategias de búsqueda y otros tipos de algoritmos fundamentales.

Los algoritmos y las estructuras de datos están estrechamente relacionados y se complementan mutuamente en el desarrollo de soluciones eficientes y efectivas en el mundo de la programación. Los algoritmos definen cómo y cuándo se accede a los datos, mientras que las estructuras de datos se encargan de dónde se ubican.

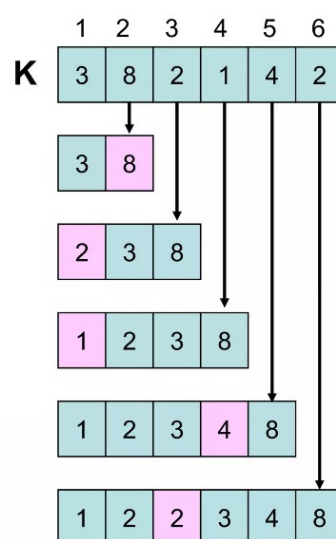
Los algoritmos especifican las operaciones, y el orden en que se realizarán, sobre los datos almacenados en las estructuras. Los algoritmos pueden ser simples o complejos, y su eficiencia y rendimiento pueden variar significativamente dependiendo de la estructura de datos utilizada. Por ejemplo, un algoritmo de búsqueda lineal puede ser adecuado para una lista desordenada, pero una búsqueda binaria será mucho más eficiente en una lista ordenada.

5.1 Algoritmos de ordenación

Empecemos el bloque de algoritmos por los de ordenación. Éstos son una tipología de procedimientos fundamentales en la informática, los cuales consisten en organizar conjuntos de datos en función de un orden o criterio específico (creciente o decreciente, alfabéticamente, etcétera). Los algoritmos de ordenación permutan los elementos del conjunto de datos hasta conseguir dicho orden. Para ello se basan en dos operaciones básicas: la comparación y el intercambio. Existen muchos algoritmos de ordenación con diferentes ventajas e inconvenientes, y en este apartado vamos a ver los más comunes.

5.1.1 Ordenación por inserción directa

Este algoritmo de ordenación consiste en lo siguiente: inicialmente, se ordenan los dos primeros elementos de la estructura de datos, luego se inserta el tercer elemento en la posición correcta respecto a los dos primeros. A continuación, se inserta el cuarto elemento en la posición correcta respecto a los tres primeros elementos ya ordenados, y así sucesivamente hasta llegar al último elemento. En la imagen del margen derecho hay un ejemplo con todos los pasos para ordenar un array inicial K.



Veamos una implementación en Java:

```
public static void ordInsercionDirecta(int[] vector) {
    int i, k = 0;
    int x;

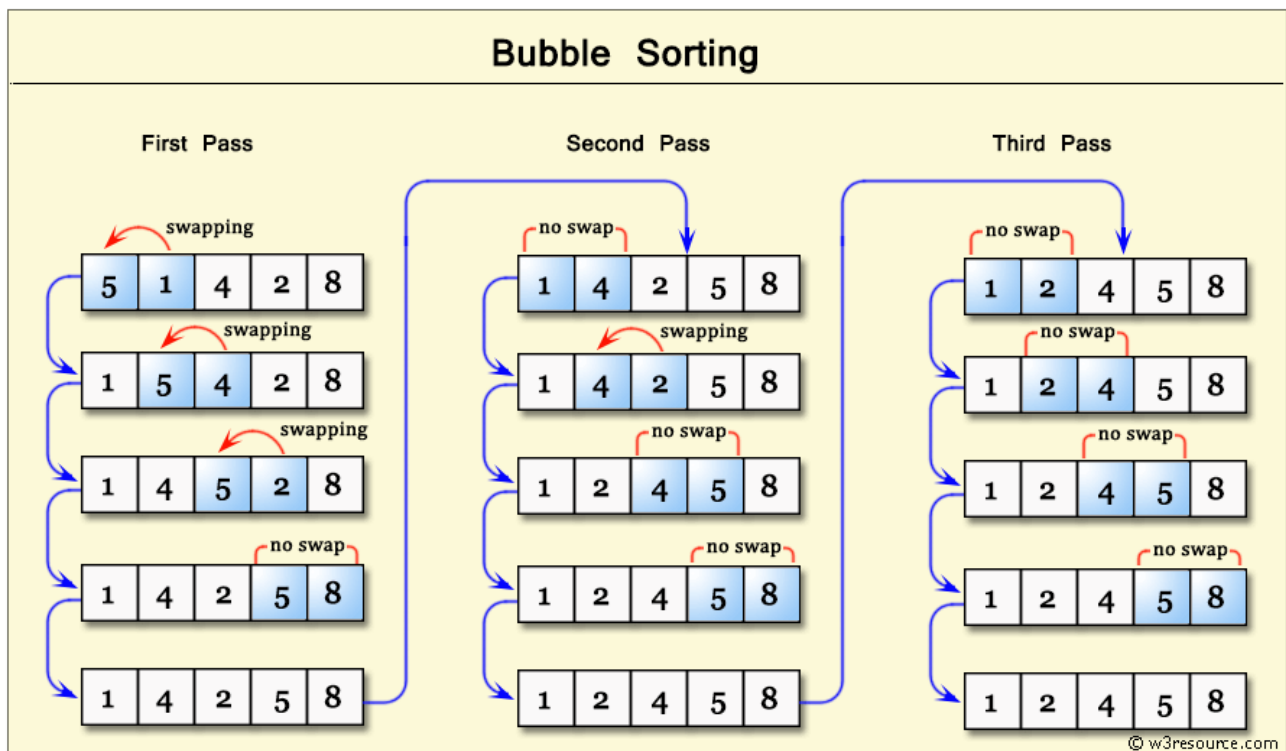
    // Empezamos desde el segundo elemento
    for (i = 1; i < vector.length; i++) {
        x = vector[i];
        k = i - 1;
        // Para k = - 1, hemos alcanzado el extremo izquierdo

        while (k >= 0 && x < matriz[k]) {
            // crea espacio para insertar
            vector[k + 1] = vector[k];
            k--;
        }
        vector[k + 1] = x; // insertamos x en su lugar
    }
}
```

5.1.2 Ordenación por el método de la burbuja

El algoritmo de la burbuja (bubble sort) es uno de los métodos de ordenación más conocidos y uno de los primeros que aprenden los programadores. Consiste en comparar pares de elementos adyacentes en un array y, si están desordenados, intercambiarlos hasta que estén todos ordenados.

```
public static void burbuja(int[] A) {  
    int i, j, aux;  
    for (i = 0; i < A.length - 1; i++) {  
        for (j = 0; j < A.length - i - 1; j++) {  
            if (A[j + 1] < A[j]) {  
                aux = A[j + 1];  
                A[j + 1] = A[j];  
                A[j] = aux;  
            }  
        }  
    }  
}
```



El tiempo de ejecución del algoritmo de la burbuja es del orden $O(n^2)$. Es uno de los peores algoritmos de ordenación en cuanto a tiempo de ejecución, solamente es recomendable su uso para ordenar listas con un número pequeño de elementos.

5.1.3 Ordenación por QuickSort

Es uno de los algoritmos de ordenación más rápidos. Se basa en la técnica **divide y vencerás**, que consiste en ir subdividiendo el array en arrays más pequeños, y ordenar éstos. Para hacer esta división, se toma un valor del array como pivote, y se mueven todos los elementos menores que este pivote a su izquierda, y los mayores a su derecha. A continuación se aplica el mismo método a cada una de las dos partes en las que queda dividido el array. Su implementación es recursiva.

```
public static void quicksort(int A[], int izq, int der) {  
  
    int pivote=A[izq]; // tomamos primer elemento como pivote  
    int i=izq;         // i realiza la búsqueda de izquierda a derecha  
    int j=der;         // j realiza la búsqueda de derecha a izquierda  
    int aux;  
  
    while(i < j){  
        while(A[i] <= pivote && i < j) i++; // busca elemento mayor que pivote  
        while(A[j] > pivote) j--;          // busca elemento menor que pivote  
        if (i < j) {                       // si no se han cruzado  
            aux= A[i];                     // los intercambia  
            A[i]=A[j];  
            A[j]=aux;  
        }  
    }  
  
    A[izq]=A[j]; // se coloca el pivote en su lugar de forma que tendremos  
    A[j]=pivote; // los menores a su izquierda y los mayores a su derecha  
  
    if(izq < j-1)  
        quicksort(A,izq,j-1); // ordenamos subarray izquierdo  
    if(j+1 < der)  
        quicksort(A,j+1,der);  // ordenamos subarray derecho  
}
```

10	40	7	9	15	27
----	----	---	---	----	----

Array original a ordenar

10	40	7	9	15	27
----	----	---	---	----	----

Se toma como pivote el primer elemento

10	40	7	9	15	27
i			j		

La búsqueda de izquierda a derecha encuentra el 40, mayor que pivote
y la búsqueda de derecha a izquierda encuentra el 9, menor que pivote

10	9	7	40	15	27
----	---	---	----	----	----

Se intercambian

10	9	7	40	15	27
	j	i			

Continúa la búsqueda, se encuentra el valor 40 mayor que pivote y el
valor 7 menor que pivote, pero ya se han cruzado. Paramos.

7	9	10	40	15	27
---	---	----	----	----	----

subarray 1

subarray 2

Finalmente colocamos el pivote en su lugar. (Posición j), quedando el
array dividido en dos subarrays a los que les aplicará el mismo proceso

5.1.4 Ordenación por MergeSort

El método MergeSort es un algoritmo de ordenación recursivo con un número de comparaciones entre elementos del array mínimo. Su funcionamiento es similar al Quicksort, y está basado en la técnica divide y vencerás. De forma resumida el funcionamiento del método MergeSort es el siguiente:

- Si la longitud del array es menor o igual a 1 entonces ya está ordenado.
- El array a ordenar se divide en dos mitades de tamaño similar.
- Cada mitad se ordena de forma recursiva aplicando el método MergeSort.
- A continuación las dos mitades ya ordenadas se mezclan formando una secuencia ordenada.

La ordenación MergeSort se puede implementar en Java de la siguiente forma:

```
public static void mergesort(int A[], int izq, int der){
    if (izq < der){
        int m=(izq+der)/2;
        mergesort(A, izq, m);
        mergesort(A, m+1, der);
        merge(A, izq, m, der);
    }
}
```

La operación principal de mezcla la realiza el método merge. Este método se puede implementar de varias formas, la más usual es la siguiente:

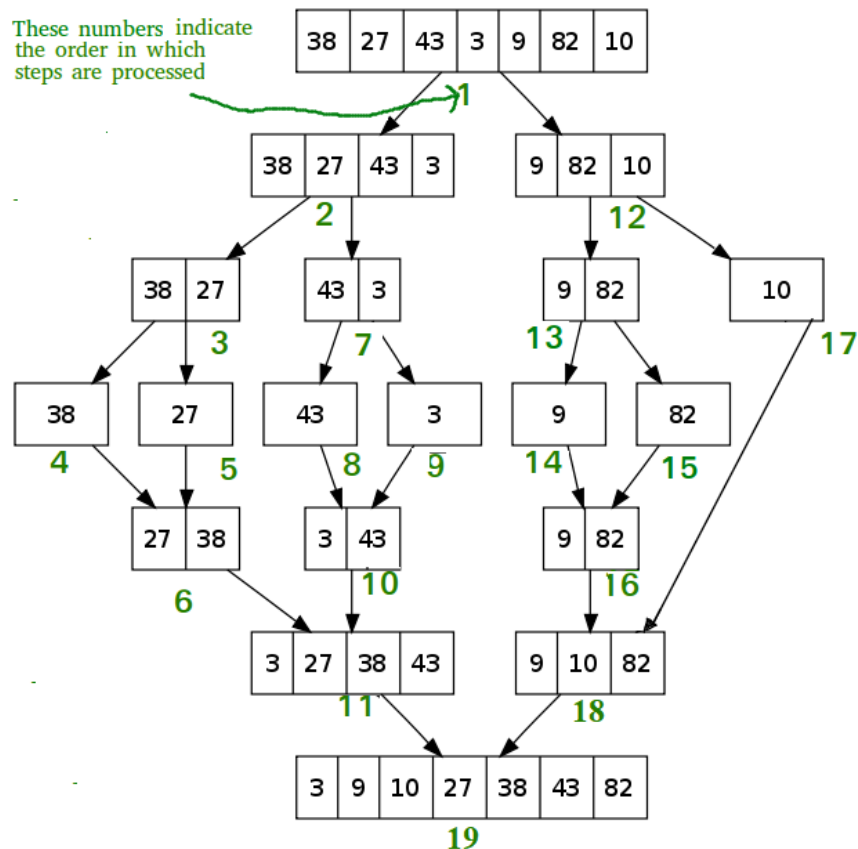
```
public static void merge(int A[], int izq, int m, int der){
    int i, j, k;
    int [] B = new int[A.length]; //array auxiliar
    for (i=izq; i<=der; i++) //copia ambas mitades en el array auxiliar
        B[i]=A[i];

    i=izq; j=m+1; k=izq;

    while (i<=m && j<=der) //copia el siguiente elemento más grande
        if (B[i]<=B[j])
            A[k++]=B[i++];
        else
            A[k++]=B[j++];

    while (i<=m) //copia los elementos que quedan de la
        A[k++]=B[i++]; //primera mitad (si los hay)
}
```

De forma gráfica podemos representar el funcionamiento del algoritmo de la siguiente forma:



El tiempo de ejecución promedio del método MergeSort es $O(n \log n)$.

5.1.5 Ordenación estándar en Java: el método .sort()

El método .sort(), internamente, implementa y usa 2 algoritmos famosos de forma combinada: quicksort y mergesort. Quicksort se usa cuando la estructura está formada por tipos primitivos, mientras que mergesort se emplea con estructuras que tienen objetos.

El método sort internamente hace una conversión explícita del objeto a la interfaz Comparable para poder comparar los objetos, sean del tipo que sean, siempre que tengan implementada esa interfaz. El parámetro del método sort indica que se puede pasar un array de objetos de cualquier tipo, es decir, arrays que almacenen objetos de tipo Empleado, Jefe, etc. Si el parámetro fuera de tipo Empleado, el método sort quedaría limitado debido a que solo podría ordenar objetos de tipo Empleado. En cuanto a la implementación del método compareTo, se recomienda que se implemente la interfaz Comparable para poder comparar objetos con el método sort.

5.1.6 Ordenación de objetos con “sort”: Comparable y Comparator

El método `.sort()` es un método de la clase `Collections` que se utiliza para ordenar estructuras de datos que implementan la interfaz `List`; sin embargo, sólo funciona si el tipo de datos posee un orden natural (como los enteros...) o bien si el objeto en cuestión tiene implementada la interfaz “Comparable”. El siguiente ejemplo funcionaría sin ningún problema:

```
List<Integer> numeros = new ArrayList<>(List.of(10, 5, 7, 1, 3));
Collections.sort(numeros);
```

Pero si tuviéramos este caso, daría un error:

```
public class Persona {
    String nombre;
    int edad;
}

ArrayList<Persona> personas = new ArrayList<>();
personas.add(new Persona("Juan", 25));
personas.add(new Persona("María", 30));
personas.add(new Persona("Pedro", 20));

Collections.sort(personas); // No funciona!!! ¿Ordeno por el nombre o la edad?
```

El problema es que “sort” busca una forma de ordenar a las personas, y esa forma no está definida ya que “Persona” es un objeto compuesto y no posee un orden natural de ordenación. En este caso, las personas podrían ser ordenadas alfabéticamente por nombre o bien numéricamente por edad. Para resolverlo, debemos definir en “Persona” una forma de ser ordenada. Veamos qué cambios realizar (en amarillo):

```
public class Persona implements Comparable<Persona> {
    String nombre;
    int edad;

    @Override
    public int compareTo(Persona p) {
        if (this.getEdad() > p.getEdad()) {
            return 1;
        } else if (this.getEdad() < p.getEdad()) {
            return -1;
        } else {
            return 0;
        }
    }
}

ArrayList<Persona> personas = new ArrayList<>();
personas.add(new Persona("Juan", 25));
personas.add(new Persona("María", 30));
personas.add(new Persona("Pedro", 20));

Collections.sort(personas); // AHORA SI!!! Ordenación por edad
```

Ahora funciona correctamente. Veamos cómo el método **compareTo** es el usado para comparar dos objetos y decidir cuando uno es mayor que otro. Devuelve 1 si el objeto actual es mayor que el que pasamos como parámetro. Devuelve 0 si en la comparación se produce una relación de igualdad y devuelve -1 si es menor.

Sin embargo, el uso de Comparable tiene limitaciones que pueden ser resueltas con el objeto **Comparator**; con Comparable **solo** podemos tener **un criterio** de **ordenación** para una clase (criterio definido en la propia clase), y con Comparator podemos ordenar por **distintos criterios**, (criterios definidos en clases independientes) y usaremos el que queramos en cada ocasión al llamar a "sort". Veamos un ejemplo:

```
public class Main {
    public static void main(String[] args) {
        List<Peli> lista = new ArrayList<>();
        lista.add(new Peli("Episode 7: The Force Awakens", 2015));
        lista.add(new Peli("Episode 4: A New Hope", 1977));
        lista.add(new Peli("Episode 1: The Phantom Menace", 1999));

        Collections.sort(lista); // NO Funciona!
        ComparaNombre compNombre = new ComparaNombre();
        Collections.sort(lista, compNombre); // Pero esto sí!!

        ComparaAnyo compAnyo = new ComparaAnyo();
        Collections.sort(lista, compAnyo);
    }
}

class Peli {
    public String nombre;
    public int año;

    public Peli(String n, int a) {
        this.nombre = n; this.año = a;
    }

    @Override
    public String toString () {
        return "(" + this.año + " " + this.nombre;
    }
}

class ComparaAnyo implements Comparator {
    public int compare(Object o1, Object o2) {
        Peli p1 = (Peli) o1; Peli p2 = (Peli) o2;
        return p1.año - p2.año;
    }
}

class ComparaNombre implements Comparator {
    public int compare(Object o1, Object o2) {
        Peli p1 = (Peli) o1; Peli p2 = (Peli) o2;
        return p1.nombre.compareToIgnoreCase(p2.nombre);
    }
}
```

Ahora, usando objetos de tipo “Comparator” ya no es necesario modificar la clase original (Peli, Persona, etc.), podemos pasar por parámetro el objeto “Comparator” al método .sort y de esa forma le estaríamos indicando cómo debe ordenar el conjunto de elementos.

Fíjate que para ordenar por Comparable hay que modificar la clase que queremos que sufra la ordenación: hay que decir que implementa Comparable y hay que sobrescribir el método compareTo(). Esto no ocurre con Comparator, no hace falta modificar la clase original a ordenar, está toda la lógica en las clases que implementan Comparator. Este puede ser otro motivo para emplear Comparator en vez de Comparable, si no podemos o no queremos modificar la clase que va a sufrir la ordenación.

5.2 Algoritmos de búsqueda

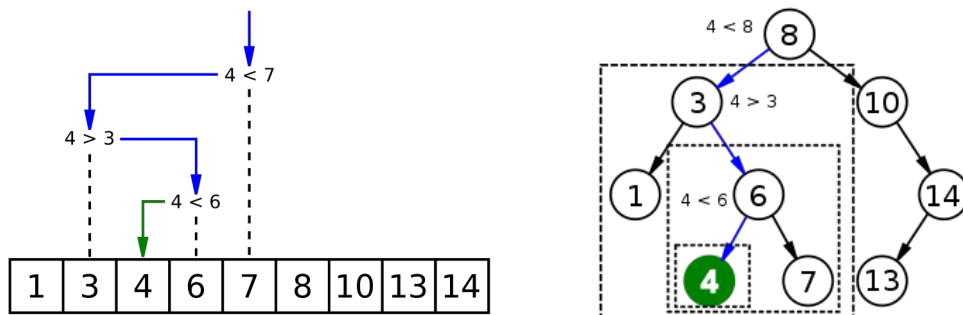
Los algoritmos de búsqueda, fundamentales en la algorítmica, son procedimientos diseñados para localizar un elemento que cumpla un patrón o condición, dentro de una estructura de datos concreta. Por ejemplo, encontrar una persona concreta en una base de datos a partir de su DNI. En este apartado estudiaremos los 2 métodos de búsqueda tradicionales: la secuencial y la binaria (o dicotómica).

5.2.1 Búsqueda secuencial

Consiste en ir comparando el elemento a buscar con cada elemento del vector hasta encontrarlo o hasta que se llegue al final, esto hace que la búsqueda sea secuencialmente (de ahí su nombre). La existencia se puede asegurar cuando el elemento es localizado, pero no podemos asegurar la no existencia hasta no haber analizado todos los elementos del vector. Se utiliza sin importar si el vector está previamente ordenado o no. Dada la sencillez del algoritmo, no pondremos aquí su implementación.

5.2.2 Búsqueda binaria (dicotómica)

Se utiliza cuando el vector en el que queremos determinar la existencia de un elemento está previamente ordenado. Este algoritmo reduce el tiempo de búsqueda considerablemente, ya que disminuye exponencialmente el número de iteraciones necesarias. En el peor de los casos el número máximo de comparaciones es $\lfloor \log_2(n) + 1 \rfloor$, donde n es el número de los elementos en el vector. Por ejemplo, en uno conteniendo 50.000.000 elementos, el algoritmo realiza como máximo 26 comparaciones.



```
import java.util.Arrays;
public class EjemploBinarySearch {

    public static void main(String[] args) {
        int[] array = { 10, 15, 20, 50, 100, 120, 200, 500, 600, 2222 };
        int result = Arrays.binarySearch(array, 400);
        System.out.println(String.format("Resultado %d", result));
    }
}
```

Es importante mencionar que el método **binarySearch** de la clase **Arrays** también requiere que el array que se pase como parámetro se encuentre ordenado, en caso contrario los resultados pueden ser erróneos. Por tanto, antes de llamar a **binarySearch** debemos ordenar la estructura de datos que se nos proporcione empleando algún método de ordenación.

5.3 Otros algoritmos

Además de los algoritmos de ordenación y búsqueda, fundamentales en el desarrollo de software, existen infinidad de algoritmos famosos de muchas otras categorías, algunos de los cuales conviene conocer. En este apartado vamos a estudiar algunos de ellos de forma conceptual:

5.3.1 Técnicas y estrategias avanzadas para diseño de algoritmos

El diseño de algoritmos es un proceso crucial en la resolución de problemas computacionales. A continuación, se presentan algunas técnicas y estrategias comunes utilizadas para diseñar algoritmos eficientes y efectivos para el problema concreto a resolver:

- Divide y vencerás (recursividad)
- Programación dinámica
- Algoritmos voraces
- Backtracking
- Ramificación y poda

Divide y vencerás (recursividad)

Este concepto ya ha ido apareciendo a medida que se avanzaba en la programación. Simplemente recordaremos que consiste en dividir el problema a resolver en subproblemas más pequeños, luego se resuelve cada subproblema de manera recursiva y, finalmente, se combinan las soluciones para obtener la solución final. **Ejemplos:** Algoritmos de ordenación como Mergesort y Quicksort utilizan esta técnica para ordenar arrays.

Programación dinámica

Frecuentemente para resolver un problema complejo se tiende a dividir este en subproblemas más pequeños, resolver estos últimos (recurriendo posiblemente a nuevas subdivisiones) y combinar las soluciones obtenidas para calcular la solución del problema inicial; es decir, aplicar la técnica de “Divide y vencerás” a través de la recursividad. Puede ocurrir que la división natural del problema conduzca a un gran número de subejemplares idénticos.

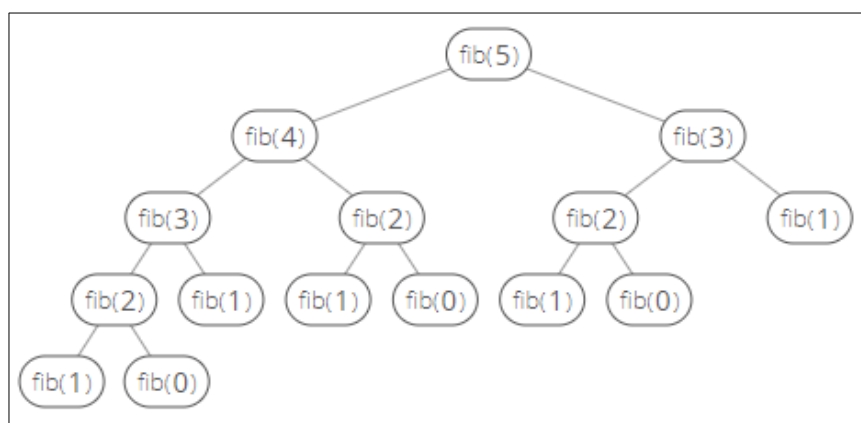
Si se resuelve cada uno de ellos sin tener en cuenta las posibles repeticiones, resulta un algoritmo ineficiente, cosa que sucede con la recursividad en el momento en que el tamaño del

problema es grande; en cambio si se resuelve cada ejemplar distinto una sola vez y se conserva el resultado, el algoritmo obtenido es mucho más eficiente.

Esta es la idea de la programación dinámica: no calcular dos veces lo mismo y utilizar normalmente una tabla de resultados que se va rellenando a medida que se resuelven los subejemplares. La programación dinámica es un método ascendente. Se resuelven primero los subejemplares más pequeños y por tanto más simples. Combinando las soluciones se obtienen las soluciones de ejemplares sucesivamente más grandes hasta llegar al ejemplar original. Sin embargo, no todo son ventajas ya que resolver problemas con programación dinámica es complicado y requiere de un análisis muy profundo del problema.

La programación dinámica y los algoritmos voraces pueden aplicarse al mismo problema, siendo especialmente indicadas para problemas de optimización; pero se diferencian en un factor: el algoritmo voraz es, generalmente, más eficiente pero no garantiza la optimalidad.

Ejemplo tradicional de la programación dinámica: calcular el número de fibonacci de forma recursiva. Fíjate en el diagrama inferior, que muestra el árbol de llamadas desde un ejemplo con el valor 5. ¿Tiene sentido calcular el valor fib(2) (y muchos otros) cada vez que aparece? No, estamos haciendo lo mismo varias veces...podemos evitarlo...



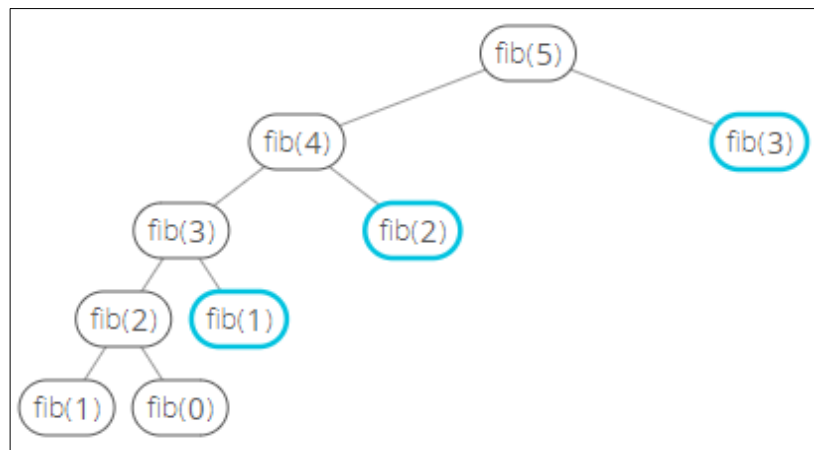
Veamos el ejemplo en Java. Primero recordemos cómo sería el método recursivo:

```
public static int fibonacci(int n){  
    if(n <= 1)  
        return n;  
    else  
        return fibonacci(n-1) + fibonacci(n-2);  
}
```

Ahora veamos cómo cambiaría para aplicar **programación dinámica**:

```
public static int fibonacci(int n, HashMap<Integer, Integer> memo) {  
    if (memo.containsKey(n)) {  
        return memo.get(n);  
    }  
  
    if (n <= 1) {  
        return n;  
    } else {  
        int result = fibonacci(n - 1, memo) + fibonacci(n - 2, memo);  
        memo.put(n, result);  
        return result;  
    }  
}
```

¿Cómo cambiaría el árbol de llamadas (y la tabla de ayuda)? Así:



Clave	0	1	2	3
Valor	0	1	1	2

Algoritmos voraces

Son algoritmos que toman decisiones de corto alcance, basadas en información inmediatamente disponible, sin importar consecuencias futuras. Suelen ser bastante simples y se emplean sobre todo para resolver problemas de optimización, como por ejemplo, encontrar la secuencia óptima para procesar un conjunto de tareas por un computador, hallar el camino mínimo de un grafo, etc. El nombre voraz proviene de que, en cada paso, el algoritmo escoge el mejor "pedazo" que es capaz de "comer" en ese momento, sin preocuparse del futuro. Nunca deshace una decisión ya tomada: una vez incorporado un elemento a la solución permanece ahí hasta el final; y cada vez que un elemento es rechazado, lo es para siempre.

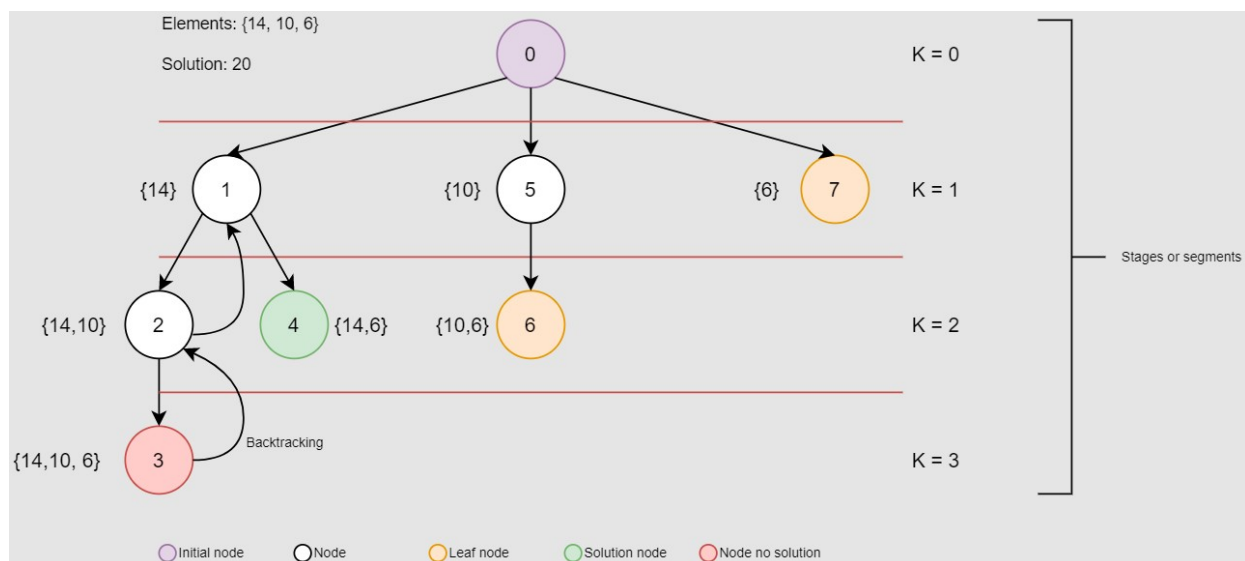
Una de sus características más destacadas, que puede ser una desventaja, es que **no garantizan la solución óptima global**: aunque los algoritmos voraces pueden producir soluciones óptimas en muchos casos, no siempre se garantiza que la solución global sea la mejor posible.

Backtracking

Técnica ampliamente utilizada para resolver problemas a través de una búsqueda exhaustiva recursiva, generalmente sobre estructuras de datos de tipo árbol. La idea es encontrar la mejor combinación posible en un momento determinado, por eso, se dice que este tipo de algoritmo es una búsqueda en profundidad. Durante la búsqueda, si se encuentra una alternativa incorrecta, se retrocede hasta el paso anterior y se toma la siguiente alternativa.

Cuando se han terminado las posibilidades, se vuelve a la elección anterior y se toma la siguiente opción (hijo [si nos referimos a un árbol]). Si no hay más alternativas la búsqueda falla. Ejemplos de aplicación: sudoku, movimientos de ajedrez, inteligencia artificial.

Dado un conjunto de números enteros {14, 10, 6} encontrar si existe algún subconjunto cuya suma sea igual a 20.



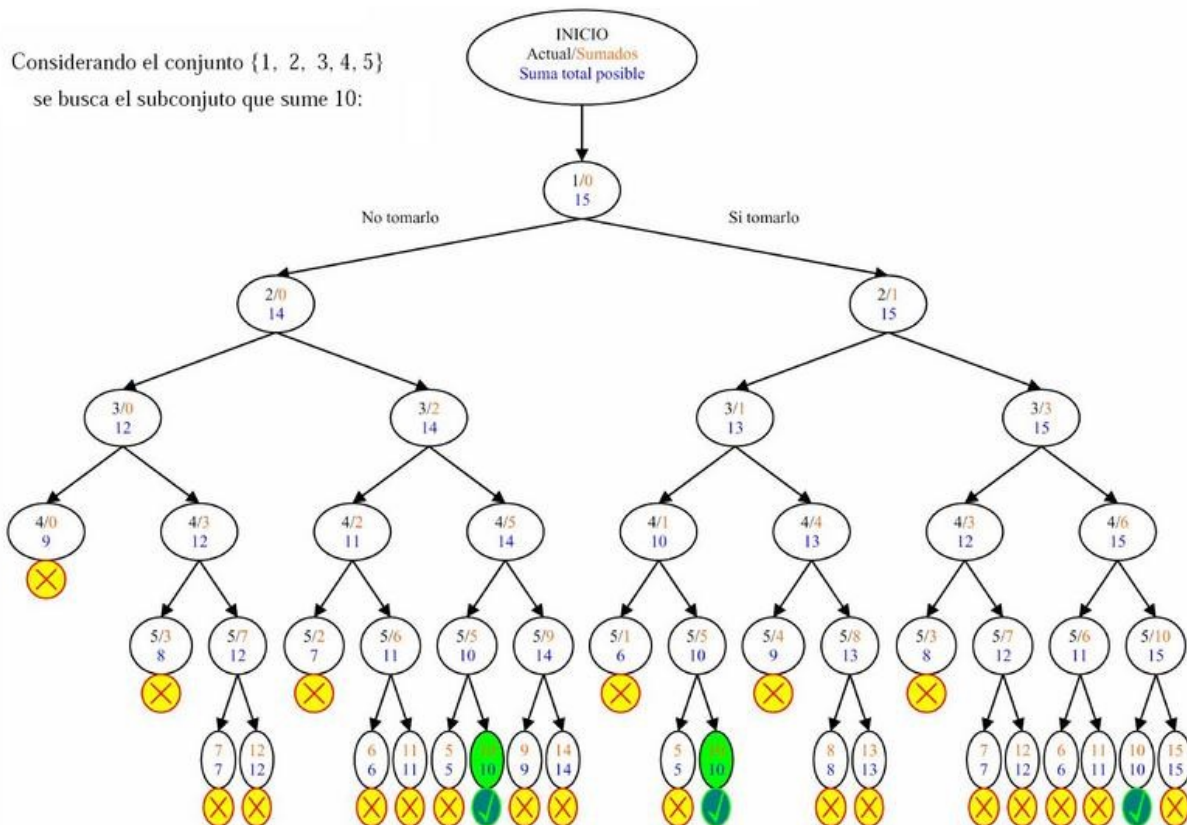
Ramificación y poda

Es una variante del Backtracking mejorado sustancialmente. El término (del inglés, Branch and Bound) se aplica mayoritariamente para resolver cuestiones o problemas de optimización.

La técnica de Ramificación y poda se suele interpretar como un árbol de soluciones, donde cada rama nos lleva a una posible solución posterior a la actual. La característica de esta técnica

con respecto a otras anteriores (y a la que debe su nombre) es que el algoritmo se encarga de detectar en qué ramificación las soluciones dadas ya no están siendo óptimas, para «podar» esa rama del árbol y no continuar malgastando recursos y procesos en casos que se alejan de la solución óptima.

El siguiente ejemplo muestra, gráficamente, el árbol de soluciones que se exploran para el problema de encontrar un subconjunto de números que sumen 10 a partir del conjunto: {1, 2, 3, 4, 5}. Fíjate en los nodos marcados en amarillo y un triángulo, marcan soluciones NO válidas; y los nodos marcados en verde, que sí son soluciones válidas.

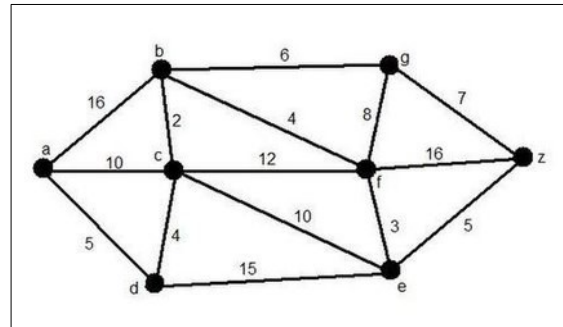


5.3.2 Algoritmos del camino más corto y recubrimiento mínimo

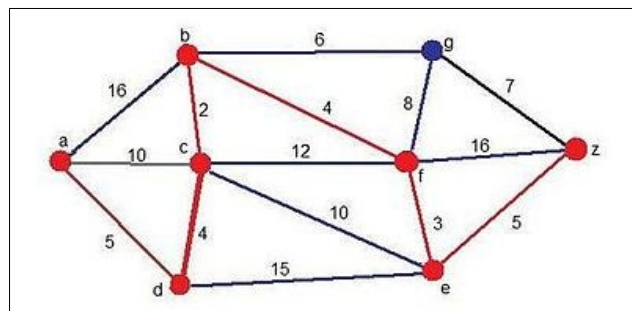
Son algoritmos muy utilizados en campos diversos, como por ejemplo: el enrutamiento en redes de comunicaciones, el cálculo de la ruta en un GPS, obtener la manera más económica de conectar por carretera varias ciudades de un recorrido, y un largo etcétera. Los más importantes son: Dijkstra, Kruskal, Prim, Bellman-Ford y Ford-Fulkerson.

Algoritmo de Dijkstra

Es un algoritmo para la determinación del camino más corto, dado un vértice origen, hacia el resto de los vértices en un grafo que tiene pesos en cada arista. Usado, por ejemplo, en el cálculo de una ruta por GPS. Veamos la ruta más corta desde “a” hasta “z” en el siguiente grafo:



Tras varios pasos, concretamente en este ejemplo harían falta 7, llegamos a la solución:



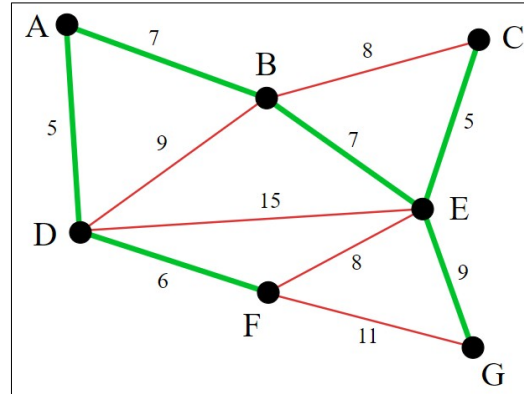
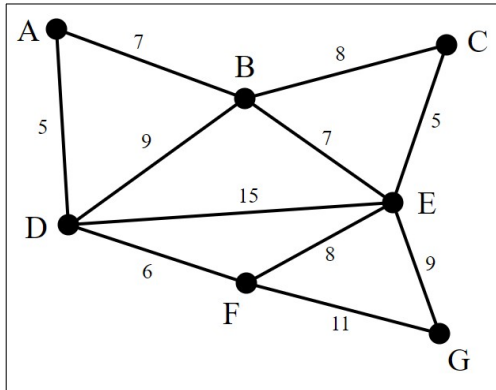
Su implementación es muy compleja y no la vamos a ver aquí.

Algoritmo de Kruskal

Partiendo de un grafo conexo, no dirigido y ponderado, encuentra un árbol de expansión mínima. Es decir, es capaz de encontrar un subconjunto de las aristas que formen un árbol que incluya todos los vértices del grafo, donde el peso total de las aristas es el mínimo posible.

Una de las aplicaciones más destacadas del árbol mínimo recubridor se encuentra en el ámbito de las telecomunicaciones, por ejemplo, en las redes de comunicación eléctrica, telefónica, etc. Los nodos representarían puntos de consumo eléctrico, teléfonos, aeropuertos o computadoras. Las aristas podrían ser cables de alta tensión, fibra óptica, rutas aéreas, etc.

Veamos un ejemplo para ayudarnos a entenderlo mejor. Supongamos que el siguiente grafo muestra varias ciudades (etiquetadas con las letras A a G) y las distancias entre ellas (en kilómetros). El algoritmo de Kruskal encontrará la forma de conectar todas las ciudades de la forma más “barata” o “corta” posible. A la izquierda el grafo original y a la derecha el final:



En verde se observa la ruta que forma parte de la solución y su distancia total es de 39 kilómetros. Si tuviéramos que unir todas las ciudades por tendido eléctrico, la cantidad mínima de cable sería de 39 kilómetros.